

Reflection Statement: Transitioning to Modern Enterprise Architectures

Project Name: Queue Master Car Wash System

Focus Area: Backend API Design, Spring Boot Autoconfiguration

Working on this project gave me practical experience in backend development using Spring Boot and helped me understand how modern web applications are structured and developed. My main role in the project was developing and implementing backend functionality, particularly user authentication and communication between the frontend and backend systems.

Role and Responsibilities

One of my major responsibilities in the project was creating REST API endpoints for user registration and login which is how the user will access our wash management system. I used Spring Boot annotations such as `@RestController`, `@PostMapping`, `@RequestMapping`, and `@Autowired` to define controllers and manage dependency injection, it creates objects automatically

Transition from Java EE to Spring Boot

In older Java EE applications, web traffic was managed using Servlets configured in large `web.xml` files, and responses were typically HTML pages generated by JSPs. This tightly coupled architecture meant the backend was responsible for both data processing and UI rendering.

Spring Boot introduced a decoupled architecture. The backend no longer needed to know about the user interface. Instead, it focused solely on processing data and returning JSON responses. The frontend, built with React, handled presentation and user interaction.

This separation, known as decoupling, provided several benefits:

- Flexibility – The frontend and backend could evolve independently.
- Scalability – Each layer could be scaled separately depending on demand.

Maintainability – Bugs and issues could be isolated to either frontend or backend without affecting the other.

`@RestController` used to define a class as a web controller meaning it returns data not views/pages

@CrossOrigin It allows your backend to accept requests from another origin.

By using the @CrossOrigin annotation, I learned why browsers block requests between different origins and how backend applications can allow trusted frontend applications to communicate with them. This gave me insight into frontend-backend integration and common issues developers face when connecting applications running on different ports or domains.

@PostMapping is a Spring Boot annotation used to handle HTTP POST requests automatically.

In Java EE, you would manually handle POST requests

Spring simplifies this using @PostMapping

@RequestBody automatically convert JSON data sent from the React frontend into a Java AppUser object.

So instead of manually reading and parsing the data, Spring Boot automatically turns the JSON into a usable object in the backend.

@Autowired it creates objects automatically

API Response Management

I learned that HTTP status codes are important for communicating what is happening in an API request. Instead of just sending plain text or redirecting to error pages, I used ResponseEntity to send proper responses which improves the user experience and simplifies frontend error handling.

for example

If a username already exists, I returned 400 (Bad Request)

If login was successful, I returned 200 (OK) along with the user data

If login failed, I returned 401 (Unauthorized)

This project was a very useful learning experience that helped me grow both technically and professionally.

I was able to use Java in a real-world project, especially by building backend authentication features using Spring Boot

It also helped me improve important skills like problem-solving, debugging, and understanding how different parts of a software system work together